

Problema Lab 5: Migrarea schemei BD și metode avansate

Obiectiv

Învățarea gestionării evoluției schemei bazei de date folosind instrumente de migrare (Liquibase sau Flyway), implementarea pattern-urilor de control al concurenței și înțelegerea strategiilor de ștergere soft.

Obiective de Învățare

- Controlul versiunilor schemei bazei de date cu instrumente de migrare
- Practicarea evoluției sigure a schemei în medii de echipă
- Implementarea locking-ului optimist pentru actualizări concurente
- Înțelegerea compromisurilor între ștergere soft și hard

Cerințe

1. Configurarea Instrumentului de Migrare

Alegeți **Liquibase** (recomandat) sau **Flyway** pentru migrările bazei de date.

Java - Configurare Liquibase:

```
<!-- pom.xml -->
<dependency>
  <groupId>org.liquibase</groupId>
  <artifactId>liquibase-core</artifactId>
  <version>4.24.0</version>
</dependency>
```

C# - Migrări EF Core:

```
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet ef migrations add InitialCreate
dotnet ef database update
```

2. Sarcini de Evoluție a Schemei

Sarcina A: Migrare Inițială

Creați migrarea inițială pentru schema existentă a bazei de date din laboratoarele anterioare:

- Exportați schema curentă în format de migrare
- Creați fișier baseline changelog/migration

- Documentați procesul

Sarcina B: Adăugare Coloană Nouă

Creați o migrare care adaugă o coloană nouă la un tabel existent:

```
-- Exemplu: Adăugați număr de telefon la angajați  
ALTER TABLE employees ADD COLUMN phone VARCHAR(20);
```

Cerințe:

- Migrarea trebuie să fie reversibilă (includeți script down/rollback)
- Adăugați valoare implicită sau gestionați NULL în mod corespunzător
- Actualizați clasele entitate pentru a reflecta schimbarea

Sarcina C: Adăugare Tabel Nou

Creați o migrare pentru un tabel nou cu relație de cheie străină:

```
-- Exemplu: Adăugați tabel proiecte  
CREATE TABLE projects (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    description TEXT,  
    start_date DATE,  
    department_id INTEGER,  
    FOREIGN KEY (department_id) REFERENCES departments(id)  
);
```

Cerințe:

- Includeți constrângeri corespunzătoare (NOT NULL, FK, etc.)
- Adăugați date de exemplu într-o migrare de date separată
- Actualizați entitățile și relațiile ORM

Sarcina D: Modificare Coloană Existentă

Creați o migrare care modifică o coloană existentă:

```
-- Exemplu: Schimbați coloana salary pentru a permite valori mai mari  
ALTER TABLE employees ALTER COLUMN salary TYPE DECIMAL(12,2);
```

Gestionați problemele de compatibilitate a datelor dacă este necesar.

Sarcina E: Adăugare Index-uri

Creați o migrare care adaugă index-uri pentru performanță:

```
CREATE INDEX idx_employees_department ON employees(department_id);
CREATE INDEX idx_employees_email ON employees(email);
```

Măsurați performanța interogărilor înainte și după adăugarea index-urilor.

3. Implementare Locking Optimist

Implementați locking optimist pentru a preveni actualizările pierdute în scenarii concurente.

Adăugați Coloană de Versiune:

Creați o migrare:

```
ALTER TABLE employees ADD COLUMN version INTEGER DEFAULT 0;
```

Actualizați Entitatea:

Java (JPA):

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Version
    private Integer version;

    // alte câmpuri...
}
```

C# (EF Core):

```
public class Employee
{
    public int Id { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; }

    // alte proprietăți...
}
```

Demonstrați Gestionarea Actualizărilor Concurente:

Creați un scenariu care arată:

1. Utilizatorul A încarcă înregistrarea angajatului (versiune = 1)
2. Utilizatorul B încarcă aceeași înregistrare (versiune = 1)

3. Utilizatorul A actualizează și salvează (versiunea devine 2)
4. Utilizatorul B încearcă să actualizeze cu versiune învechită (versiune = 1)
5. Sistemul detectează conflictul și arată eroare corespunzătoare

Cerințe:

- Implementați gestionare corectă a excepțiilor pentru conflicte de versiune
- Afișați mesaj de eroare prietenos pentru utilizator explicând conflictul
- Oferiți opțiuni: reîncărcarea datelor, actualizare forțată sau anulare
- Înregistrați conflictul pentru debugging

4. Implementare Ștergere Soft

Implementați funcționalitatea de ștergere soft ca alternativă la ștergerile hard.

Adăugați Coloană Ștergere Soft:

Migrare:

```
ALTER TABLE employees ADD COLUMN is_deleted BOOLEAN DEFAULT FALSE;
ALTER TABLE employees ADD COLUMN deleted_at TIMESTAMP;
ALTER TABLE employees ADD COLUMN deleted_by VARCHAR(100);
```

Actualizați Entitatea:

Adăugați câmpuri pentru ștergere soft și suprascrieți comportamentul de ștergere:

Java:

```
@Entity
@Where(clause = "is_deleted = false") // Annotare Hibernate
public class Employee {
    private Boolean isDeleted = false;
    private LocalDateTime deletedAt;
    private String deletedBy;

    // Metodă ștergere soft
    public void softDelete(String username) {
        this.isDeleted = true;
        this.deletedAt = LocalDateTime.now();
        this.deletedBy = username;
    }
}
```

C#:

```
public class Employee
{
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public string DeletedBy { get; set; }
```

```

}

// În DbContext, adăugați filtru global de interogare
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>()
        .HasQueryFilter(e => !e.IsDeleted);
}

```

Implementați Funcționalități:

- Ștergere soft: marchează înregistrarea ca ștearsă dar o păstrează în baza de date
- Listare înregistrări șterse (vedere admin)
- Restaurare înregistrări șterse
- Opțiune ștergere permanentă (doar pentru administratori)
- Comparatie: arătați diferența de comportament între ștergere soft și hard

Cerințe:

- Implicit, interogările NU ar trebui să returneze înregistrări șterse soft
- Oferiți interfață admin pentru vizualizarea înregistrărilor șterse
- Implementați funcționalitatea de restaurare
- Adăugați audit trail (cine a șters, când)

5. Livrabile

1. **Cod sursă** cu toate migrările și implementările
2. **Fișiere de migrare** în format corespunzător (changelog.xml, fișiere .sql sau migrări EF)
3. **Diagramă schemă bază de date** arătând schimbările înainte/după migrare
4. **Video demonstrativ sau capturi de ecran** arătând:
 - Rularea migrărilor înainte și înapoi
 - Gestionarea conflictelor de locking optimist
 - Operații de ștergere soft și restaurare
5. **Raport cuprinzător (2-4 pagini)** acoperind:
 - Strategia de migrare aleasă și rațiunea
 - Documentație pas cu pas a execuției migrărilor
 - Explicația locking-ului optimist și rezultatele demonstrației
 - Comparatie ștergere soft vs hard și compromisuri
 - Lecții învățate și best practices
 - Provocări întâmpinate

8. Criterii de Evaluare (100 puncte)

- **Implementare Migrări (35 puncte)**
 - Toate sarcinile de migrare completate corect (20)
 - Funcționalitate rollback demonstrată (10)
 - Best practices pentru migrări urmate (5)
- **Locking Optimist (25 puncte)**
 - Implementare corectă cu coloană de versiune (15)
 - Detectare și gestionare corespunzătoare a conflictelor (10)

- **Ștergere Soft (25 puncte)**
 - Implementare completă ștergere soft (15)
 - Funcționalitate restaurare (5)
 - Vedere admin pentru înregistrări șterse (5)
- **Documentație (15 puncte)**
 - Raport complet cu analiză (10)
 - Ghid best practices migrări (5)

9. Puncte Bonus (+15)

- Implementați seeding-ul bazei de date cu migrări (+5)
- Creați o migrare personalizată pentru a migra date între coloane (+5)
- Implementați audit logging pentru toate schimbările (created_by, updated_by, etc.) (+5)

Predare

- **Termen limită:** [Va fi anunțat]